C++ Tutorial

Zhu Bicen

Fasttrack Cplane Support Team

June 19, 2012

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

the first function example

```
1  // function example
2  int addition (int a, int b)
3  {
4    int r;
5    r=a+b;
6    return (r);
7  }
8  int main ()
9  {
10   int z;
11   z = addition (5,3);
12   cout << "The result is " << z;
13   return 0;
14 }
```

Arguments passed by value and by reference

```
1  void duplicate (int& a, int& b, int& c)
2  {
3     a*=2;
4     b*=2;
5     c*=2;
6  }
7
8  int main ()
9  {
10    int x=1, y=3, z=7;
11    duplicate (x, y, z);
12    cout << "x=" << x << ", y=" << y << ", z=" << z;
13    return 0;
14 }
```

Default values in parameters

```
1  int divide (int a, int b=2)
2  {
3     int r;
4     r=a/b;
5     return (r);
6  }
7
8  int main ()
9  {
10    cout << divide (12);
11    cout << endl;
12    cout << divide (20,4);
13    return 0;
14 }
```

Overloaded functions I

```
1 int operate (int a, int b)
2 {
3   return (a*b);
4 }
5
6 float operate (float a, float b)
7 {
8   return (a/b);
9 }
10
11
12 int main ()
13 {
14   int x=5,y=2;
```

Overloaded functions II

```
15    float n=5.0,m=2.0;
16    cout << operate (x,y);
17    cout << "\n";
18    cout << operate (n,m);
19    cout << "\n";
20    return 0;
21 }
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

Accessing the values of an array

```
1 int main ()
2 {
3   int billy [] = {16, 2, 77, 40, 12071};
4   int result = 0;
5   for ( int n=0 ; n<5 ; n++ )
6   {
7     result += billy[n];
8   }
9   cout << result;
10 }
```

Multidimensional arrays

```
1  const int WIDTH 5
2  const int HEIGHT 3
3
4  int jimmy [HEIGHT][WIDTH];
5  int n,m;
6
7  int main ()
8  {
9    for (n=0;n<HEIGHT;n++)
10     for (m=0;m<WIDTH;m++)
11     {
12       jimmy[n][m]=(n+1)*(m+1);
13     }
14   return 0;
15 }
```

Arrays as parameters

```
1  void printarray (int arg[], int length) {
2    for (int n=0; n<length; n++)
3      cout << arg[n] << "␣";
4    cout << "\n";
5  }
6
7  int main ()
8  {
9    int firstarray[] = {5, 10, 15};
10   int secondarray[] = {2, 4, 6, 8, 10};
11   printarray (firstarray,3);
12   printarray (secondarray,5);
13   return 0;
14 }
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

Reference operator(&)

The address that locates a variable within memory is what we call a reference to that variable. This reference to a variable can be obtained by preceding the identifier of a variable with an ampersand sign (&), known as reference operator, and which can be literally translated as "address of".

```
1 int i;
2 int* pint = &i;
```

Dereference operator (*)

Using a pointer we can directly access the value stored in the variable which it points to. To do this, we simply have to precede the pointer's identifier with an asterisk (*), which acts as dereference operator and that can be literally translated to "value pointed by".

```
1 int i;
2 int* pint = &i;
3 int j = *pint;
```

Declaring variables of pointer types

```
1  int main ()
2  {
3    int firstvalue, secondvalue;
4    int * mypointer;
5
6    mypointer = &firstvalue;
7    *mypointer = 10;
8    mypointer = &secondvalue;
9    *mypointer = 20;
10   cout << "firstvalue is " << firstvalue << endl;
11   cout << "secondvalue is " << secondvalue << endl;
12   return 0;
13 }
```

an example a little bit more elaborated I

```
1  int main ()
2  {
3     int firstvalue = 5, secondvalue = 15;
4     int * p1, * p2;
5
6     p1 = &firstvalue;   // p1 = address of firstvalue
7     p2 = &secondvalue;  // p2 = address of secondvalue
8     *p1 = 10;           // value pointed by p1 = 10
9     *p2 = *p1;          // value pointed by p2 =
10                        //value pointed by p1
11    p1 = p2;            // p1 = p2 (value of pointer i
12    *p1 = 20;           // value pointed by p1 = 20
13
14    cout << "firstvalue␣is␣" << firstvalue << endl;
```

an example a little bit more elaborated II

```
15    cout << "secondvalue␣is␣" << secondvalue << endl;
16    return 0;
17 }
```

Pointers and Arrays

```
1  int main ()
2  {
3    int numbers[5];
4    int * p;
5    p = numbers;  *p = 10;
6    p++;  *p = 20;
7    p = &numbers[2];  *p = 30;
8    p = numbers + 3;  *p = 40;
9    p = numbers;  *(p+4) = 50;
10   for (int n=0; n<5; n++)
11     cout << numbers[n] << ", ";
12   return 0;
13 }
```

Pointer arithmetics

Pointers to pointers

```
1 char a;
2 char * b;
3 char ** c;
4 a = 'z';
5 b = &a;
6 c = &b;
```

Null pointer

A null pointer is a regular pointer of any pointer type which has a special value that indicates that it is not pointing to any valid reference or memory address. This value is the result of type-casting the integer value zero to any pointer type.

```
int* p;
p = 0;//nullptr
```

Pointers to functions I

```
1 int addition (int a, int b)
2 { return (a+b); }
3
4 int subtraction (int a, int b)
5 { return (a.b); }
6
7 int operation (int x, int y, int
8 (*functocall)(int,int))
9 {
10   int g;
11   g = (*functocall)(x,y);
12   return (g);
13 }
14
```

Pointers to functions II

```
15 int main ()
16 {
17   int m,n;
18   int (*minus)(int,int) = subtraction;
19
20   m = operation (7, 5, addition);
21   n = operation (20, m, minus);
22   cout <<n;
23   return 0;
24 }
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

Operators new and new[]

```
1 int * bobby;
2 bobby = new int [5];
3 int * bobby;
4 bobby = new (nothrow) int [5];
5 if (bobby == 0) {
6   // error assigning memory. Take measures.
7   };
```

Defined data types (typedef)

```
1 typedef char C;
2 typedef unsigned int WORD;
3 typedef char * pChar;
4 typedef char field [50];
```

Enumerations(enum)

```
1 enum colors_t {
2   black, blue, green,
3   cyan, red, purple,
4   yellow, white};
5
6 colors_t mycolor;
7
8 mycolor = blue;
9 if (mycolor == green) mycolor = red;
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

complete example of class CRectangle I

```
1  class CRectangle {
2      int x, y;
3    public:
4      void set_values (int,int);
5      int area () {return (x*y);}
6  };
7  void CRectangle::set_values (int a, int b) {
8    x = a;
9    y = b;
10 }
11 int main () {
12   CRectangle rect;
13   rect.set_values (3,4);
14   cout << "area: " << rect.area();
```

complete example of class CRectangle II

```
15    return 0;
16 }
```

Constructors and destructors I

```
1 class CRectangle {
2     int *width, *height;
3   public:
4     CRectangle (int,int);
5     ~CRectangle ();
6     int area () {return (*width * *height);}
7 };
8 CRectangle::CRectangle (int a, int b) {
9   width = new int;
10  height = new int;
11  *width = a;
12  *height = b;
13 }
14 CRectangle::~CRectangle () {
```

Constructors and destructors II

```
15    delete width;
16    delete height;
17 }
18 int main () {
19    CRectangle rect (3,4), rectb (5,6);
20    cout << "rect area: " << rect.area() << endl;
21    cout << "rectb area: " << rectb.area() << endl;
22    return 0;
23 }
```

Overloading Constructors I

```
1 class CRectangle {
2     int width, height;
3   public:
4     CRectangle ();
5     CRectangle (int,int);
6     int area (void) {return (width*height);}
7 };
8
9 CRectangle::CRectangle () {
10   width = 5;
11   height = 5;
12 }
13
14 CRectangle::CRectangle (int a, int b) {
```

Overloading Constructors II

```
15    width = a;
16    height = b;
17 }
18
19 int main () {
20    CRectangle rect (3,4);
21    CRectangle rectb;
22    cout << "rect area: " << rect.area() << endl;
23    cout << "rectb area: " << rectb.area() << endl;
24    return 0;
25 }
```

Default constructor

```
1 class CExample {
2   public:
3     int a,b,c;
4     void multiply (int n, int m) { a=n; b=m; c=a*b;
5   };
```

Overloading operators I

```
1  class CVector {
2    public:
3      int x,y;
4      CVector () {};
5      CVector (int,int);
6      CVector operator + (CVector);
7  };
8
9  CVector::CVector (int a, int b) {
10   x = a;
11   y = b;
12 }
13
14 CVector CVector::operator+ (CVector param) {
```

Overloading operators II

```
15    CVector temp;
16    temp.x = x + param.x;
17    temp.y = y + param.y;
18    return (temp);
19 }
20
21 int main () {
22    CVector a (3,1);
23    CVector b (1,2);
24    CVector c;
25    c = a + b;
26    cout << c.x << "," << c.y;
27    return 0;
28 }
```

The keyword this I

```
1  class CDummy {
2    public:
3      int isitme (CDummy& param);
4  };
5
6  int CDummy::isitme (CDummy& param)
7  {
8    if (&param == this) return true;
9    else return false;
10 }
11
12 int main () {
13   CDummy a;
14   CDummy* b = &a;
```

The keyword this II

```
15    if ( b.>isitme(a) )
16      cout << "yes, &a is b";
17    return 0;
18 }
```

static members

```
1 class CDummy {
2   public:
3     static int n;
4     CDummy () { n++; };
5     ~CDummy () { n--; };
6 };
7
8 int CDummy::n=0;
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

Friend functions I

```
1 class CRectangle {
2     int width, height;
3   public:
4     void set_values (int, int);
5     int area () {return (width * height);}
6     friend CRectangle duplicate (CRectangle);
7 };
8
9 void CRectangle::set_values (int a, int b) {
10   width = a;
11   height = b;
12 }
13
14 CRectangle duplicate (CRectangle rectparam)
```

Friend functions II

```
15 {
16   CRectangle rectres;
17   rectres.width = rectparam.width*2;
18   rectres.height = rectparam.height*2;
19   return (rectres);
20 }
```

Inheritance between classes I

```
class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b;}
  };

class CRectangle: public CPolygon {
  public:
    int area ()
      { return (width * height); }
  };

```

Inheritance between classes II

```
15 class CTriangle: public CPolygon {
16   public:
17     int area ()
18       { return (width * height / 2); }
19   };
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

Pointers to base class I

```
class CPolygon {
  protected:
    int width, height;
  public:
    void set_values (int a, int b)
      { width=a; height=b; }
  };

class CRectangle: public CPolygon {
  public:
    int area ()
      { return (width * height); }
  };

```

Pointers to base class II

```
15 class CTriangle: public CPolygon {
16   public:
17     int area ()
18       { return (width * height / 2); }
19   };
20 int main () {
21   CRectangle rect;
22   CTriangle trgl;
23   CPolygon* ppoly1 = &rect;
24   CPolygon* ppoly2 = &trgl;
25   ppoly1->set_values (4,5);
26   ppoly2->set_values (4,5);
27   cout << rect.area() << endl;
28   cout << trgl.area() << endl;
29 }
```

Virtual members I

```
1  class CPolygon {
2    protected:
3      int width, height;
4    public:
5      void set_values (int a, int b)
6        { width=a; height=b; }
7      virtual int area ()
8        { return (0); }
9    };
10
11 class CRectangle: public CPolygon {
12   public:
13     int area ()
14       { return (width * height); }
```

Virtual members II

```
15   };
16
17 class CTriangle: public CPolygon {
18   public:
19     int area ()
20       { return (width * height / 2); }
21   };
22
23 int main () {
24   CRectangle rect;
25   CTriangle trgl;
26   CPolygon poly;
27   CPolygon* ppoly1 = &rect;
28   CPolygon* ppoly2 = &trgl;
29   CPolygon* ppoly3 = &poly;
```

Virtual members III

```
30    ppoly1->set_values (4,5);
31    ppoly2->set_values (4,5);
32    ppoly3->set_values (4,5);
33    cout << ppoly1->area() << endl;
34    cout << ppoly2->area() << endl;
35    cout << ppoly3->area() << endl;
36    return 0;
37 }
```

Abstract base classes

```
1 class CPolygon {
2   protected:
3     int width, height;
4   public:
5     void set_values (int a, int b)
6       { width=a; height=b; }
7     virtual int area () =0;
8 };
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

Function templates

```
1  template <class T>
2  T GetMax (T a, T b) {
3    T result;
4    result = (a>b)? a : b;
5    return (result);
6  }
7  int main () {
8    int i=5, j=6, k;
9    long l=10, m=5, n;
10   k=GetMax<int>(i,j);
11   n=GetMax<long>(l,m);
12   cout << k << endl;
13   cout << n << endl;
14   return 0;
15 }
```

class template I

```
1  template <class T>
2  class mypair {
3      T a, b;
4    public:
5      mypair (T first, T second)
6        {a=first; b=second;}
7      T getmax ();
8  };
9
10 template <class T>
11 T mypair<T>::getmax ()
12 {
13   T retval;
14   retval = a>b? a : b;
```

class template II

```
15    return retval;
16 }
17
18 int main () {
19    mypair <int> myobject (100, 75);
20    cout << myobject.getmax();
21    return 0;
22 }
```

Template specialization I

```
// class template:
template <class T>
class mycontainer {
    T element;
  public:
    mycontainer (T arg) {element=arg;}
    T increase () {return ++element;}
};
// class template specialization:
template <>
class mycontainer <char> {
    char element;
  public:
    mycontainer (char arg) {element=arg;}
```

Template specialization II

```
15      char uppercase ()
16      {
17        if ((element>='a')&&(element<='z'))
18        element+='A'.'a';
19        return element;
20      }
21 };
22 int main () {
23   mycontainer<int> myint (7);
24   mycontainer<char> mychar ('j');
25   cout << myint.increase() << endl;
26   cout << mychar.uppercase() << endl;
27   return 0;
28 }
```

Non-type parameters for templates

```
1 template <class T, int N>
2 class mysequence {
3     T memblock [N];
4   public:
5     void setmember (int x, T value);
6     T getmember (int x);
7 };
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

```
1  namespace first
2  {
3     int var = 5;
4  }
5
6  namespace second
7  {
8     double var = 3.1416;
9  }
10
11 int main () {
12    cout << first::var << endl;
13    cout << second::var << endl;
14    return 0;
15 }
```

using I

```
1  namespace first
2  {
3     int x = 5;
4     int y = 10;
5  }
6
7  namespace second
8  {
9     double x = 3.1416;
10    double y = 2.7183;
11 }
12
13 int main () {
14    using first::x;
```

using II

```
15    using second::y;
16    cout << x << endl;
17    cout << y << endl;
18    cout << first::y << endl;
19    cout << second::x << endl;
20    return 0;
21 }
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

```
1  int main () {
2    try
3    {
4      throw 20;
5    }
6    catch (int e)
7    {
8      cout << "An exception occurred. "
9      cout << "Exception Nr. " << e << endl;
10   }
11   return 0;
12 }
```

Standard exceptions I

```
1  class myexception: public exception
2  {
3    virtual const char* what() const throw()
4    {
5      return "My exception happened";
6    }
7  } myex;
8
9  int main () {
10   try
11   {
12     throw myex;
13   }
14   catch (exception& e)
```

Standard exceptions II

```
15    {
16       cout << e.what() << endl;
17    }
18    return 0;
19 }
```

Outline

1. Control Structures
   - Function
2. Compound data types
   - Arrays
   - Pointers
   - Dynamic Memory
3. Object Oriented Programming
   - Class
   - Friendship and Inheritance
   - Polymorphism
4. Advanced concepts
   - Templates
   - Namespace
   - Exception
   - Type Casting
5. STL

Implicit conversion

```
1 short a = 2000;
2 int b;
3 b = a;
4
5
6 class A {};
7 class B { public: B (A a) {} };
8
9 A a;
10 B b=a;
```

Explicit conversion

```
short a=2000;
int b;
b = (int) a;      // c.like cast notation
b = int (a);      // functional notation
```

dynamic_cast I

```
1  class CBase { virtual void dummy() {} };
2  class CDerived: public CBase { int a; };
3
4  int main () {
5    try {
6      CBase * pba = new CDerived;
7      CBase * pbb = new CBase;
8      CDerived * pd;
9
10     pd = dynamic_cast<CDerived*>(pba);
11     if (pd==0) cout << "Null␣pointer␣on␣first␣type.
12
13     pd = dynamic_cast<CDerived*>(pbb);
14     if (pd==0) cout << "Null␣pointer␣on␣second␣type
```

dynamic_cast II

```
15
16    } catch (exception& e) {cout << "Exception: " << e
17    return 0;
18 }
```

static_cast

```
1 class CBase {};
2 class CDerived: public CBase {};
3 CBase * a = new CBase;
4 CDerived * b = static_cast<CDerived*>(a);
5
6 double d=3.14159265;
7 int i = static_cast<int>(d);
```

reinterpret_cast

```
1 void* malloc(size_t);
2 void f()
3 {
4   int* p = static_cast<int*>(malloc(100));
5   IO_Device* dl = reinterpret_cast<IO_device*>(0xff
6 }
```

6.2.7 The static_cast operator converts between related types such as one pointer type to another, an umeration to an integral type, or a floating-point type to an integral type. The reinterpret_cast conversions between unrelated types such as an integer to a pointer.

const_cast

```
1 void print (char * str)
2 {
3   cout << str << endl;
4 }
5
6 int main () {
7   const char * c = "sample␣text";
8   print ( const_cast<char *> (c) );
9   return 0;
10 }
```

Thanks!